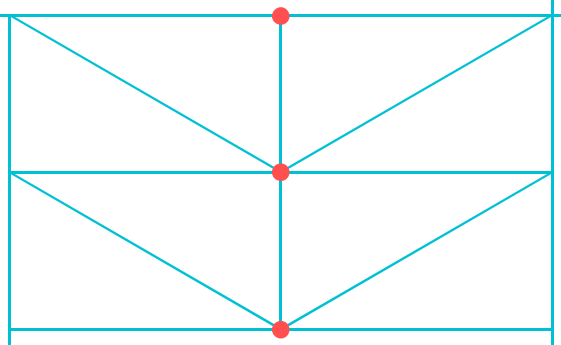
Distributed Systems

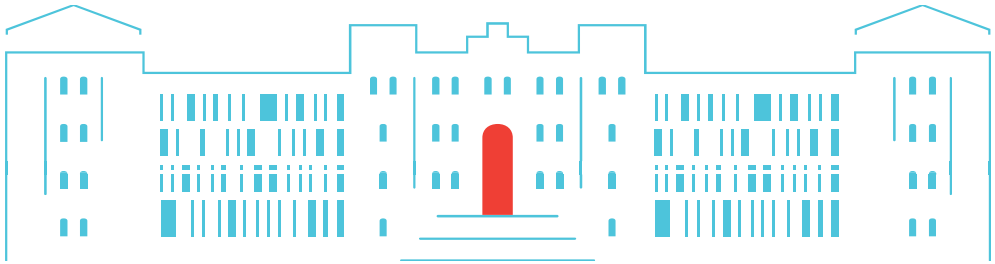
Lab 6

Fault Tolerance

TUHH
Technische
Universität
Hamburg



14.1.26 and 16.1.26



Marcus Venzke

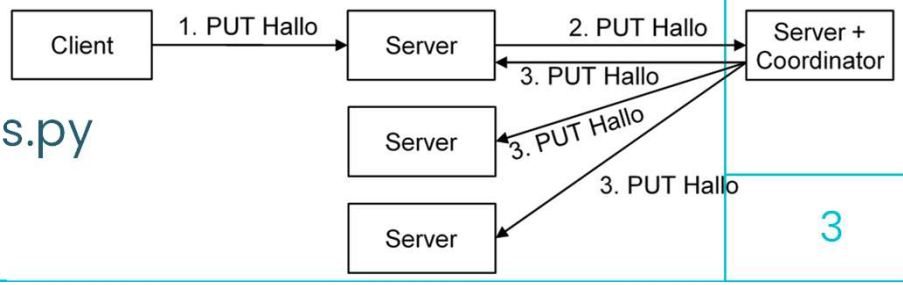
Agenda

- Two tasks about two issues of fault tolerance:
- Task 1: A server crashes:
  Cluster works. Eventual consistency after restart
  - Checkpointing
  - Store updates of crashed server in queue
- Task 2: No errors if message lost on the network.
  Remote Procedure calls with:
  - At-most-once semantics
  - At-least-once semantics for non-idempotent functions

TUHH

Task 1: Handle Server Crashes

- Goal: A server crashes
  - Cluster continues to work. Also updates possible
  - Eventual consistency after server is restarted
- Requires:
  - a) Checkpointing ⇒ No data lost after restart
    - Save message board in after updates
    - Load last checkpoint on restart
  - b) Updates for crashed server stored in queue
    - ⇒ Eventual consistency after restart
- We use SendToCoordinatorAndBackToServers.py
- Our approach is not perfect!



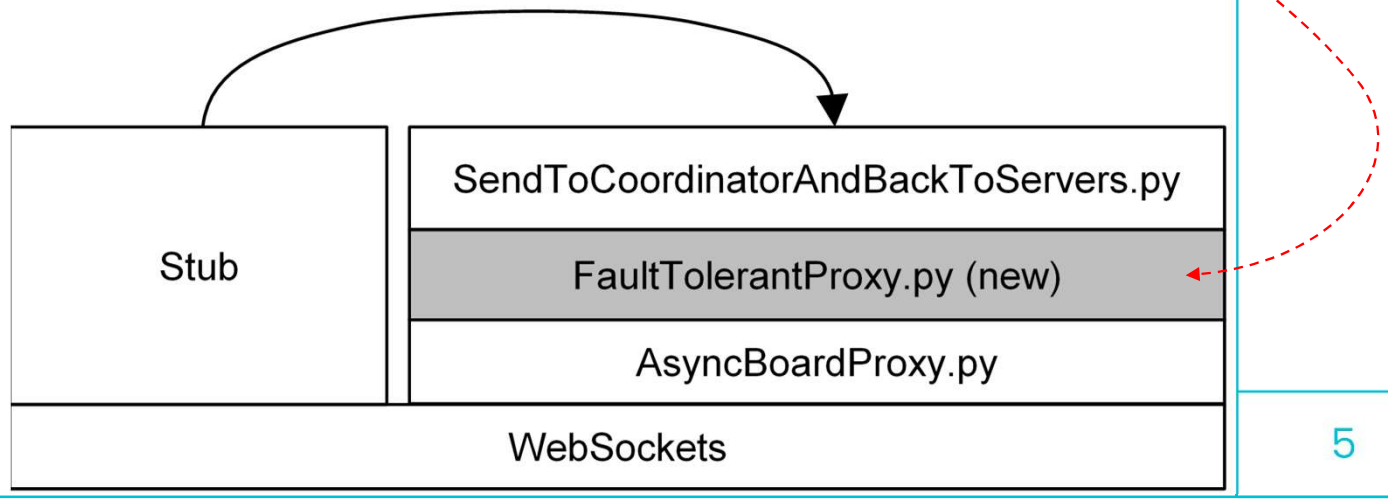
Task 1 a) – Checkpointing

- Extend AsyncBoardStorage.py to do checkpointing
  - In case of updates (put(), modify(), delete(), deleteAll()): write message board in to JSON file
    - Filename include server's ID (every server needs own checkpoint)
  - Constructor reads checkpoint (if available)
  - Parameter added to enable / disable checkpointing
    - For compatibility with other message board versions
- Approach: Simple, unreliable, inefficient

TUHH

Task 1 b) Queue updates for (crashed) servers

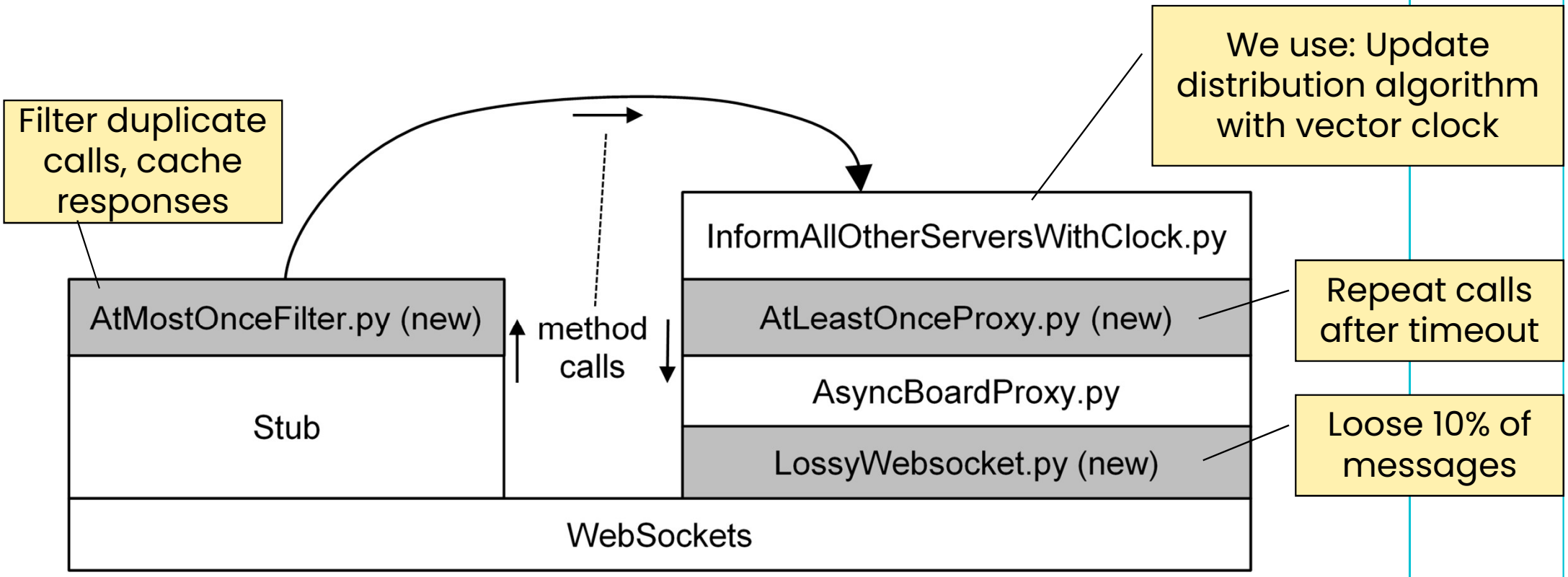
- Each server has one FCFS queue for each other server
  - Stores update operations ((put(), modify(), delete(), deleteAll()))
  - Coroutine: Periodically try: Connect to server and do updates
- Implemented in new module FaultTolerantProxy
  - Same interface as AsyncBoardProxy
  - Uses AsyncBoardProxy for update operations



Task 2: No errors if message lost on network

- Issue: WebSeckets do not lose messages
  - We add module LossyWebsocket.py losing 10% or requests + responses
  - Interface similar to WebSockets API (a subset)
  - Other function connect() given to AsyncBoardProxy as parameter
- Message loss with remote procedure calls
  - Remote procedure call hangs when message is lost
  - At-most-once semantics: Repeat remote procedure calls after timeout
  - At-least-once semantics: Avoid multiple calls of procedures on servers
    - After repeated (request) call because response was lost
    - Repeated request:
      - Do not call procedure again.
      - Respond with response from previous call (cache)

Task 2: Architecture



- You get `..._main.py` files to create and assemble all objects

Task 2 b) – Timeouts of Coroutines

- Use asyncio.timeout to detect that procedure will not terminate
- Example: Coroutine function f()

```
async with asyncio.timeout(3):
    return await f()
```

- After 3 seconds: `asyncio.TimeoutError` exception raised
  - Handle this exception
- In `AtLeastOnceProxy.py`:
  - Call operations of `AsyncBoardProxy.py` via variable `self.proxy` (hangs if request or response lost)
  - Use timeout of 3 seconds
  - Exception ⇒ Use loop to repeat call
    - At most 5 times.
      Otherwise: Infinite loop if server permanently not available

TUHH

Test your solution!!!

- Exercise sheet explains what to test
  - Fell free to also test other test cases
- Task 1: Test without and with failures
  - Simpler vs. challenging failure case
- Task2: Test with 2 update distribution algorithms
  - InformAllOtherServersWithClock.py (with vector clock)
  - SendToCoordinatorAndBackToServers.py (used in Task 1)
    - You will observe issues.

Exercise Sheet

- Tasks described in detail on exercise sheet 6
- Download exercise sheet from StudIP

- You may start implementing Lab 6 after next slide
  - You can submit solution of Lab 5
  - I will also answer questions to Lab 6

Next Week / In Two Weeks

- Next week: Time for questions
  - You may already submit your solutions
- Last week of semester:
  - Last option to submit your solution of lab 6

The End